Practice 3 - Requirement Analysis of Teedy

> Deadline: Week 4.
>
> Evaluation: onsite during the lab session.

In this practice, we'll analyze the requirements of Teedy.

First, run and play with Teedy **as an end user**. You could refer to Teedy's readme to better understand its features.

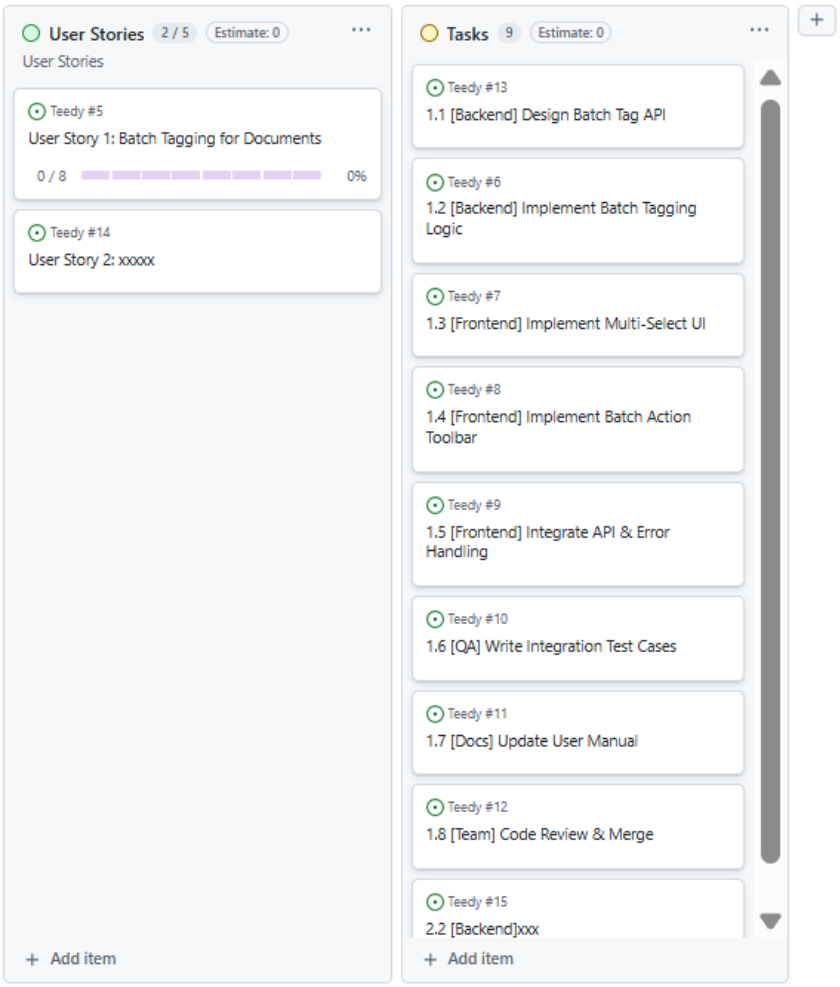
```
Responsive user interface
Optical character recognition
LDAP authentication New!
Support image, PDF, ODT, DOCX, PPTX files
Video file support
Flexible search engine with suggestions and highlighting
Full text search in all supported files
All Dublin Core metadata
Custom user-defined metadata New!
Workflow system New!
256-bit AES encryption of stored files
File versioning New!
Tag system with nesting
Import document from email (EML format)
Automatic inbox scanning and importing
User/group permission system
2-factor authentication
Hierarchical groups
Audit log
Comments
Storage quota per user
Document sharing by URL
RESTful Web API
Webhooks to trigger external service
Fully featured Android client
Bulk files importer (single or scan mode)
Tested to one million documents
```

After experiencing Teedy **as an end user**, please do the following:

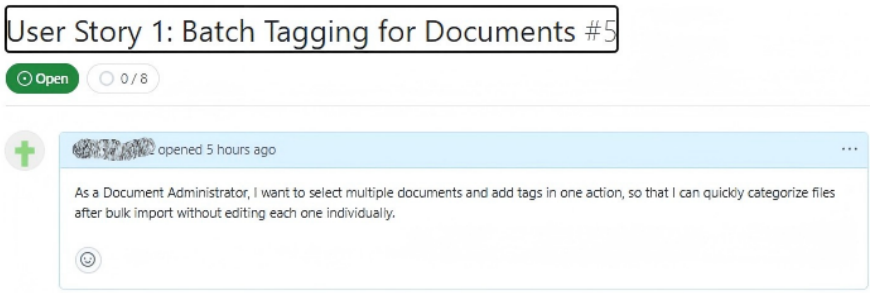
- Propose **2 new features** as an **end user**.
- As a **Product Owner**, interview your partner (acting as an end user) to **gather his\her features**, then converts these 2 features into **user stories**.
- Create a *GitHub Projects* board on the Teedy repo.
- Add all user stories to a column on the board.
- Select **at least two user stories**. For each user story, further break it down into a list of **actionable tasks**.

- Add these tasks into another column on the board.

For evaluation, please show us the board on your Teedy repo, which should look like this:



Your stories should be written in **lecture format (As a... I want... so that...)**.



Your tasks should follow the Product Owner's task breakdown guidelines:

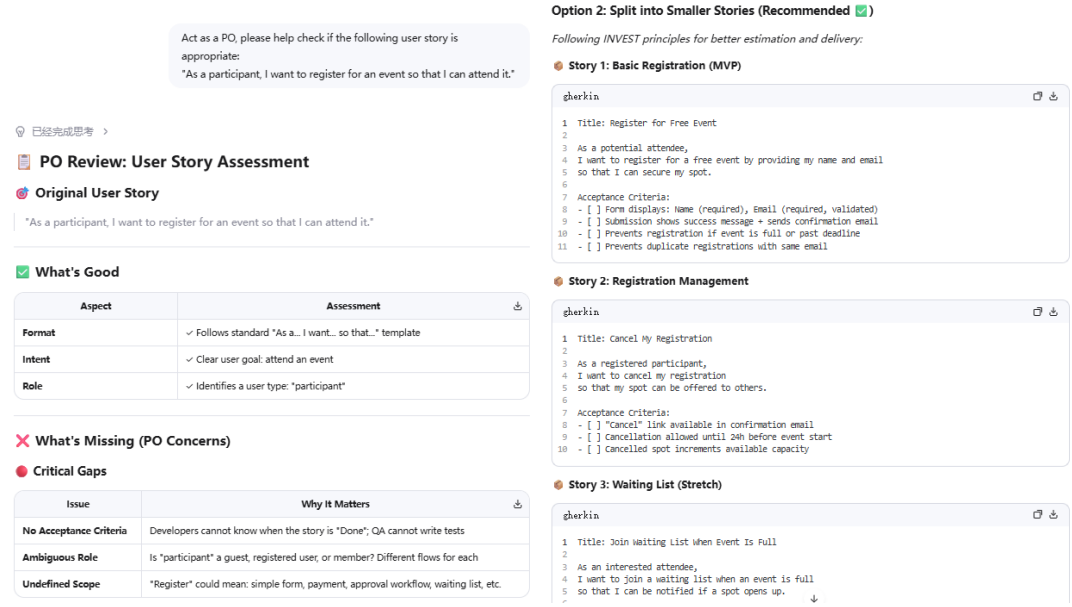
- Tasks are appropriately sized (not too big, not too small)
- Each task assigned to a single role/person
- Duration: half-day to one day maximum
- Includes both functional and non-functional tasks(e.g.;docs, testing, review)
- Each task is testable

Tips: Boost Productivity with AI

Using AI as an Agile Copilot can significantly speed up your workflow, improve quality, and ensure

consistency. However, remember: AI suggests, humans decide. You (as the Product Owner) are ultimately responsible for the value delivered.

Example 1:



Act as a PO, please help check if the following user story is appropriate:
"As a participant, I want to register for an event so that I can attend it."

已经完成思考

PO Review: User Story Assessment

Original User Story

"As a participant, I want to register for an event so that I can attend it."

What's Good

Aspect	Assessment
Format	✓ Follows standard "As a... I want... so that..." template
Intent	✓ Clear user goal: attend an event
Role	✓ Identifies a user type: "participant"

What's Missing (PO Concerns)

Critical Gaps

Issue	Why It Matters
No Acceptance Criteria	Developers cannot know when the story is "Done"; QA cannot write tests
Ambiguous Role	Is "participant" a guest, registered user, or member? Different flows for each
Undefined Scope	"Register" could mean: simple form, payment, approval workflow, waiting list, etc.

Option 2: Split into Smaller Stories (Recommended ✅)

Following INVEST principles for better estimation and delivery:

Story 1: Basic Registration (MVP)

```gherkin
Title: Register for Free Event

As a potential attendee,
I want to register for a free event by providing my name and email
so that I can secure my spot.

Acceptance Criteria:
- [ ] Form displays: Name (required), Email (required, validated)
- [ ] Submission shows success message + sends confirmation email
- [ ] Prevents registration if event is full or past deadline
- [ ] Prevents duplicate registrations with same email
```

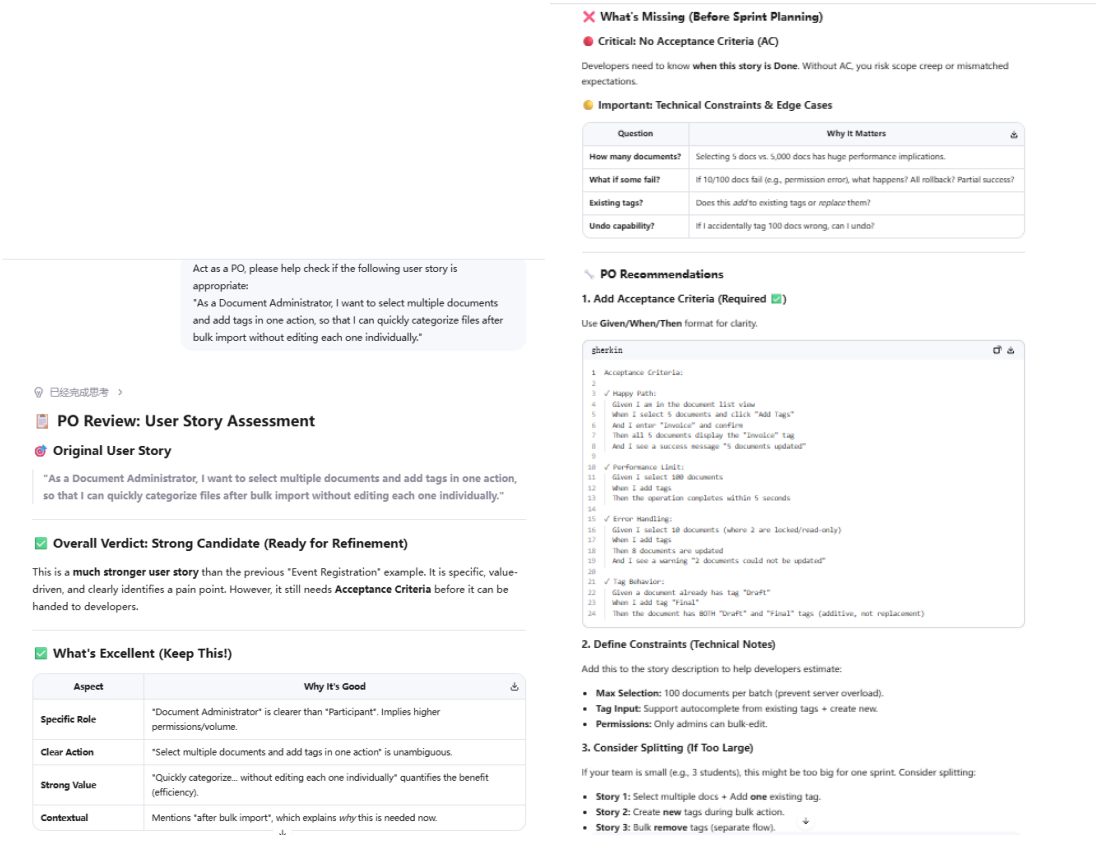
Story 2: Registration Management

```gherkin
Title: Cancel My Registration

As a registered participant,
I want to cancel my registration
so that my spot can be offered to others.

Acceptance Criteria:
- [ ] "Cancel" link available in confirmation email
- [ ] Cancellation allowed until 24h before event start
- [ ] Cancelled spot increments available capacity
```

Story 3: Waiting List (Stretch)

```gherkin
Title: Join Waiting List When Event Is Full

As an interested attendee,
I want to join a waiting list when an event is full
so that I can be notified if a spot opens up.
```

Example 2:



Act as a PO, please help check if the following user story is appropriate:
"As a Document Administrator, I want to select multiple documents and add tags in one action, so that I can quickly categorize files after bulk import without editing each one individually."

已经完成思考

PO Review: User Story Assessment

Original User Story

"As a Document Administrator, I want to select multiple documents and add tags in one action, so that I can quickly categorize files after bulk import without editing each one individually."

Overall Verdict: Strong Candidate (Ready for Refinement)

This is a **much stronger user story** than the previous "Event Registration" example. It is specific, value-driven, and clearly identifies a pain point. However, it still needs **Acceptance Criteria** before it can be handed to developers.

What's Excellent (Keep This!)

Aspect	Why It's Good
Specific Role	"Document Administrator" is clearer than "Participant". Implies higher permissions/volume.
Clear Action	"Select multiple documents and add tags in one action" is unambiguous.
Strong Value	"Quickly categorize... without editing each one individually" quantifies the benefit (efficiency).
Contextual	Mentions "after bulk import", which explains *why* this is needed now.

What's Missing (Before Sprint Planning)

Critical: No Acceptance Criteria (AC)

Developers need to know **when this story is Done**. Without AC, you risk scope creep or mismatched expectations.

Important: Technical Constraints & Edge Cases

Question	Why It Matters
How many documents?	Selecting 5 docs vs. 5,000 docs has huge performance implications.
What if some fail?	If 10/100 docs fail (e.g., permission error), what happens? All rollback? Partial success?
Existing tags?	Does this *add* to existing tags or *replace* them?
Undo capability?	If I accidentally tag 100 docs wrong, can I undo?

PO Recommendations

1. Add Acceptance Criteria (Required ✅)

Use **Given/When/Then** format for clarity.

```gherkin
Acceptance Criteria:

✓ Happy Path:
  Given I am in the document list view
  When I select 5 documents and click "Add Tags"
  And I enter "invoice" and confirm
  Then all 5 documents display the "invoice" tag
  And I see a success message "5 documents updated"

✓ Performance Limit:
  Given I select 100 documents
  When I add tags
  Then the operation completes within 5 seconds

✓ Error Handling:
  Given I select 10 documents (where 2 are locked/read-only)
  When I add tags
  Then 8 documents are updated
  And I see a warning "2 documents could not be updated"

✓ Tag Behavior:
  Given a document already has tag "Draft"
  When I add tag "Final"
  Then the document has BOTH "Draft" and "Final" tags (additive, not replacement)
```

2. Define Constraints (Technical Notes)

Add this to the story description to help developers estimate:

- **Max Selection:** 100 documents per batch (prevent server overload).
- **Tag Input:** Support autocomplete from existing tags + create new.
- **Permissions:** Only admins can bulk-edit.

3. Consider Splitting (If Too Large)

If your team is small (e.g., 3 students), this might be too big for one sprint. Consider splitting:

- **Story 1:** Select multiple docs + Add **one** existing tag.
- **Story 2:** Create **new** tags during bulk action.
- **Story 3:** Bulk **remove** tags (separate flow).